Knowledge Distillation and Deployment of a Command-Parsing LM on a Local Kubernetes Cloud

Shiv Pratap Singh - 230101097 - https://github.com/shiv-pratap04/cloud-kd-project

Measured results come from my own runs (logs in `results/`). Assumptions and interpretations are labelled as such.

0. Architecture and decisions

A phi3.5 teacher (Ollama on the Windows host, temperature 0, seed 42) labels 3,000 commands in two parallel worker pods (namespace `pipeline`). The workers read from and write to SeaweedFS (namespace `data`, buckets `datasets`, `models`) on a single-node k3d cluster. `clean.py` filters the replies into `datasets/v1/train.jsonl`. Pythia-70M is fine-tuned on the first 1,500 cleaned records (sequence-level distillation) on CPU. Weights go to `models/v1`, `models/v2` and to GitHub Releases. A FastAPI service in a CPU-only image downloads the model at start-up and runs in namespace `app` behind a Service, a Traefik Ingress and an HPA. GitHub Actions lints, tests, evaluates the released model and gates on JSON validity.

Teacher settings (disclosure). Besides temperature 0 and seed 42, I set `num_predict=80` and a stop sequence at the first newline. In a first test, one reply continued past the JSON with an explanatory note (16 s instead of 5 s); the stop sequence removed this. The prompt itself was not changed.

1. Task 1 - Object storage and parallel labelling

Setup. `cluster/k3d.yaml`; SeaweedFS in `data`; buckets `datasets` and `models`; an Indexed Kubernetes Job in `pipeline` runs two workers at the same time, with `SHARD_INDEX` 0 and 1 (`results/task1_pods.txt`). Ollama ran on the laptop; pods reach it through the WSL gateway address, because `host.k3d.internal` (the Docker gateway) refused connections from pods.

Sharding. A worker takes the commands whose 0-based line number n satisfies n mod `SHARD_COUNT` = `SHARD_INDEX`. Every line has exactly one remainder, so each command lands in exactly one shard with no coordination between workers. Each worker writes only its own object (`teacher/shard-<i>.jsonl`), so there are no write conflicts, and it checkpoints every 25 records and resumes after a restart. *Limitation:* both workers share one CPU-bound Ollama slot, so a second worker adds concurrency but not throughput.

Measured. Shard check (`results/task1_shard_check.txt`): pool 3,000, shard-0 1,500, shard-1 1,500, overlap 0, missing 0, repeats 0. Labelling took 3 h 38 min.

input	(a) not JSON	(b) wrong keys	(c) bad intent	(d) slot not in command	duplicates	kept
3000	0	3	5	294	43	2655

Check: 0+3+5+294+43+2655 = 3000. SHA-256 of `train.jsonl`: `c90f3acec7ca31ecf05a1c38a16a1ba8f84147b27c0b9f3b3a6d698d83429019`; re-running `clean.py` offline on the shards gave the same value as `train.sha256`.

2. Task 2 - Student training (sequence-level distillation)

Setup. `training/config.yaml`: seed 42, learning rate 5e-5, batch size 8, 3 epochs, max length 128, torch 2.4.1, transformers 4.44.2. Each example is `Command: <text>\nJSON: <teacher reply>`; the loss is computed on the reply tokens only. `make train` reads only that file; `make train SUBSET=50` ran to the end (losses 1.1819, 0.2825, 0.1627). The seed is fixed in the code.

Measured training loss. v1 (seed 42, 3 epochs): 0.2558, 0.0991, 0.0772. v2 (seed 7, 4 epochs): 0.2557, 0.1141, 0.0871, 0.0587.

Artefacts. `models/v1/` and `models/v2/` each hold 8 files in object storage. Release asset SHA-256: v1 `e64bd394474b806715fe85e810cf1093b65bbc5253eb74b11edcfff0cf658dd0`, v2 `27982776a5ec99d0f53fe5389ca23bd5bf4ada883e37ef021269e64d80cd1f59`; each matches its `model.sha256` after downloading the asset again.

Validation results (100 commands, measured).

model	json_validity	intent	date	time	place	full_frame
teacher phi3.5	99	85	69	85	88	44
Pythia-70M, untrained	0	0	0	0	0	0
student v1	100	77	72	89	86	41

Discussion. Training took the model from no valid JSON to valid JSON for every validation command, and the student's full-frame score (41) is close to the teacher's (44). It is better than the teacher on date and time and worse on intent. *Interpretation (not tested):* `clean.py` removed teacher replies with slot values not copied from the command, so the student only saw well-formed slots; with 100 examples, differences of a few points are within noise. I did not train a logit-based model, so I cannot say how it would compare.

Sequence-level vs logit-based distillation. Sequence-level distillation trains the student with cross-entropy on the text the teacher generated; the teacher is only a label generator. Logit-based distillation trains the student to match the teacher's per-token probability distribution (for example with a KL-divergence loss), which carries more information than the one chosen token. It is not possible here because phi3.5 and Pythia use different tokenizers and vocabularies, so their distributions cannot be aligned token by token, and I only collected the teacher's text. A real limitation of my approach: the student inherits the teacher's mistakes (the teacher itself reached only 44% full-frame, and 294 of its replies failed the slot-copy rule), and 1,500 examples from one prompt cover few phrasings.

3. Task 3 - Inference service

`service/app.py` implements `POST /parse` (HTTP 200, four-key JSON only, headers `X-Model-Version` and `X-Served-By`), `GET /healthz`, and `GET /readyz` (503 until the model is loaded). Configuration comes from `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, `MAX_NEW_TOKENS`; credentials come from a Kubernetes Secret. Every output is validated against the schema rules; an invalid output gives HTTP 500 with `{"error":"invalid_model_output"}`. The image uses pinned versions, the CPU-only torch wheel and no weights.

Measured. Image size 1.69 GB. With `FAULT_INJECT=1`, `/parse` returned 500 with exactly `{"error":"invalid_model_output"}` and both headers; `/healthz` returned 200 and `/readyz` 503 while loading, then 200 (`results/task3_api_tests.txt`). At `--cpus=1 --memory=1g`, the container was ready after about 16 s; 50 sequential requests all returned 200 (median 0.35 s, max 0.77 s); peak memory 379 MiB (17 samples at 1 s intervals, so the true peak may be slightly higher); `OOMKilled: false`.

4. Task 4 - Kubernetes deployment, scaling, reliability

Manifests: `k8s/30-app.yaml` (2 replicas, requests 500m / 512Mi, limits 1 CPU / 1Gi, startup, liveness and readiness probes, `maxUnavailable: 0`, a 15 s `preStop` sleep, a 45 s grace period), a Service, a Traefik Ingress for `/parse`, and an HPA (1 to 2 replicas, 70% CPU).

experiment	measured result
Endpoints while model loads	`kubectl get endpoints` empty at 5 s; two pod IPs after readiness
HPA	events: "New size: 2 (cpu above target)", later "New size: 1 (all metrics below target)"; `results/hpa_timeline.txt` shows 1 replica at 02:07:04 UTC, CPU 195% and 2 replicas by 02:07:33
Rolling update v1 to v2, run 1 (before fix)	1,808 requests, 13 failed (0.72%): 10 x HTTP 502, 1 x HTTP 500, 2 x connection error
Rolling update, run 2 (after `preStop` + grace period)	1,828 requests, 1 failed (0.05%, one HTTP 500, cause not determined)
Version headers	v1, then v2 after the update, then v1 after `rollout undo`
Pod deleted under load	ready pods 2, then 1 for 5 s of samples, then 2 again: restored after 8 s; 738 requests, 0 failed
Cold start (3 runs, all pods deleted)	6 s, 6 s, 6 s; median 6 s; logs show 7 files downloaded

	from `v1/` then `model ready`
Load balancing (2 Ready replicas, standard command)	267 and 269 successful requests (49.8% / 50.2%)

The run-1 failures match requests reaching a pod that was already terminating (502s from the Ingress); the fix is my interpretation, and run 2 is consistent with it. Run 1 is kept as `results/rolling_*_run1_before_fix.*`. My first recovery test read a stale `readyReplicas` value (0 s recovery) and was replaced by a version that counts ready, non-terminating pods.

Limitations of the local cluster. One node on one laptop: there is no fault tolerance across machines (if the laptop fails, everything fails), the replicas and the load generator compete for the same CPU, and the HPA can only go from 1 to 2. Cold start excludes image-pull time (the image was on the node) and the object store is inside the cluster, so a real cloud cold start would be slower. The HPA first reported metrics errors while metrics-server started.

5. Task 5 - Subnetting and NACL

Allocation from 10.20.0.0/16, largest first:

Subnet	Network	Prefix	Usable range	Broadcast	Usable hosts
app (1000)	10.20.0.0	/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public (250)	10.20.4.0	/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data (100)	10.20.5.0	/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management (20)	10.20.5.128	/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

`network/nacl.json` passes 6 of 6 sample packets in `nacl_sim.py` (measured). A NACL is stateless: it keeps no record of connections, so a reply is a separate packet and needs its own rule (outbound ports 1024-65535 to clients, inbound 1024-65535 from the app subnet). Rules run from the lowest number and the first match wins, so the deny for 198.51.100.0/24 is rule 90, below the allows on ports 80 and 443; with a higher number the allow would match first and the block would never apply. One consequence of the spec: replies to clients on port 8000 are allowed, because 8000 lies inside the ephemeral range.

6. Task 6 - Reproducibility, testing, CI/CD

`make up` from a fresh clone (one command, 12 min 9 s) built the image and deployed the system, and `/parse` returned HTTP 200 with `X-Model-Version: v1`; `make down` deleted the cluster (`results/task6_*`). A first attempt failed because my 180 s rollout timeout was too short for first-time image pulls (likely cause; not checked in detail); I raised it to 900 s. The workflow runs lint, 4 unit tests, an evaluation of the Release model with `score.py`, and the quality gate `--min-validity 95`; gitleaks reported nothing. CI first failed because the runner's default Python had no torch 2.4.1 CPU wheel; I pinned Python 3.12.

- Passing run (all stages green): https://github.com/shiv-pratap04/cloud-kd-project/actions/runs/42182377902
- Failing run, untrained Pythia-70M, fails at the quality gate (predict and score steps succeeded): https://github.com/shiv-pratap04/cloud-kd-project/actions/runs/42182679773

AI use is described in `AI_USE.md`. `make up` assumes Docker, k3d, kubectl, curl and Python with boto3 are installed.

7. Task 7 - Performance, cost, review

Standard Locust run (4 users, 2 min, through the Ingress; measured). 533 requests, 0 failures (0%), median 880 ms, p95 1,400 ms, 4.59 requests per second. `kubectl top pods` at 60 s: 966m CPU / 628Mi and 969m CPU / 652Mi; the HPA showed 210% of

target. Cold start: 6 s (Task 4).

Cost (rates from `rate_card.json`: 3.50 per vCPU-hour, 0.45 per GiB-hour, INR; each replica billed for its limits). Hourly cost of 2 replicas = 2 x (1 x 3.50 + 1 x 0.45) = 7.90. Requests per hour = 4.587 x 3600 = 16,513. Cost per 1,000 requests = 7.90 / 16,513 x 1,000 = **0.4784 INR**. This is an estimate; no cloud charge was incurred.

SLO. “99% of requests succeed” over one load-test run. Error budget = 1% x 533 = 5.33, i.e. 5 requests. Used: 0 failures = 0% of the budget.

Review.

Component	Evidence (measured)	Limitation found	Improvement
Data and storage	2,655 of 3,000 teacher replies kept; 294 rejected for slots not in the command	Single SeaweedFS pod on one volume; teacher labels are noisy	Replicated storage; filter or repair slot errors instead of dropping
Model quality	Student full-frame 41 vs teacher 44; validity 100%	Only 100 validation examples, intent 77%	Larger validation set; more distinct training data
Latency and throughput	p50 880 ms, p95 1,400 ms, 4.59 req/s	Pods hit about 970m of 1,000m CPU with only 4 users	Quantised or batched inference; more CPU per pod
Resource utilisation	379 MiB peak (1 CPU container); 628-652Mi per pod under load	Peak sampled once per second	Finer sampling; lower memory request
Reliability and recovery	Pod loss restored in 8 s with 0 of 738 failed; rolling update 0.05% failed after the fix (0.72% before)	One node; one unexplained HTTP 500	Multi-node cluster; log the 500’s cause
Security and cost	0.4784 INR per 1,000 requests; gitleaks clean	Secrets generated locally in `.env`; static S3 keys; unauthenticated `/parse`	A secret manager; short-lived credentials; API authentication